

Desafio: Organizando o Projeto da Loja Lions

Turma: LionsDev

Tópicos: navegação (`pwd` , `ls` , `cd`), organização de pastas (`mkdir` , `mv` , `cp` , `rm`), nomes com espaço, curingas (`*`), `npm init` , `npm install` , scripts do `package.json` e execução com `node` .

Pré-requisito: ter feito a lista `codigo_terminal.md` e o desafio *Explorando o Espaço*. Todo o desafio é feito **só no terminal**, exceto o passo 7, que edita o `package.json` no VS Code.

1. Contexto

Um colega começou o projeto da **Loja Lions** e salvou tudo na mesma pasta, sem organização nenhuma: códigos, anotações, imagens e arquivos temporários misturados. Sua missão é arrumar a bagunça, instalar as dependências e deixar o projeto rodando.

2. Preparação: crie a bagunça

Copie e cole este bloco no terminal. Ele cria a pasta `loja-lions` no estado em que o colega deixou:

```
mkdir loja-lions
cd loja-lions
echo 'console.log("Loja Lions no ar!");' > index.js
echo 'export function somar(a, b) { return a + b; }' > calculo.js
echo "Ideias para a loja" > anotacoes.txt
echo "arroz, feijão, café" > "lista de compras.txt"
touch logo.png banner.png foto-produto.png
touch rascunho.tmp backup.tmp
mkdir "pasta nova" antigo
touch antigo/versao-velha.js
```

Confira o resultado com `ls` . Repare que `lista de compras.txt` e `pasta nova` têm espaços no nome.

3. Missão

Execute cada passo **de dentro da pasta** `loja-lions`, a menos que o passo diga o contrário.

Passo 1: Reconhecer o terreno

Descubra em que pasta você está e liste tudo o que existe nela.

Passo 2: Criar a estrutura

Crie as pastas `src`, `docs` e `imagens`.

Passo 3: Mover as imagens de uma vez

Mova **todos** os arquivos `.png` para a pasta `imagens` usando **um único comando**.

Conceito novo: o curinga `*`. No terminal, `*` significa "qualquer sequência de caracteres". O padrão `*.png` casa com todos os arquivos que terminam em `.png`. Assim, `mv *.png imagens/` move todos de uma vez.

Passo 4: Organizar código e documentos

1. Mova `index.js` e `calculo.js` para `src`.
2. Mova `anotacoes.txt` para `docs`.
3. Mova `lista de compras.txt` para `docs` e, no mesmo comando, renomeie para `lista_compras.txt`.

Passo 5: Limpar o que não serve

1. Apague todos os arquivos `.tmp` com um único comando.
2. Apague a pasta `antigo` e tudo o que tem dentro.
3. Renomeie a pasta `pasta nova` para `config`.

Passo 6: Transformar em projeto Node

1. Crie o `package.json` aceitando as respostas padrão.
2. Instale o pacote `prompt-sync`.
3. Liste a pasta e explique o que são o `node_modules` e o `package-lock.json`, que apareceram sozinhos.
4. Mostre no terminal o conteúdo do `package.json` e encontre onde o `prompt-sync` foi registrado.

Passo 7: Rodar o projeto

1. Rode o `src/index.js` com o Node. A saída deve ser `Loja Lions no ar!`.
2. Abra o `package.json` no VS Code (`code package.json`) e, dentro de `"scripts"`, adicione o script `"start": "node src/index.js"`.
3. Rode o projeto com `npm start`.

Conceito novo: scripts do npm. O campo `"scripts"` do `package.json` guarda atalhos para comandos. Com `"start": "node src/index.js"`, qualquer pessoa que baixar o projeto roda tudo com `npm start`, sem precisar saber qual é o arquivo principal. Nos próximos módulos, é assim que as APIs serão iniciadas.

Passo 8: Conferir

Volte uma pasta acima de `loja-lions` e liste o conteúdo de todas as subpastas de uma vez com `ls -R loja-lions`.

4. Estrutura final esperada

```
loja-lions/
├── config/
├── docs/
│   ├── anotacoes.txt
│   └── lista_compras.txt
├── imagens/
│   ├── banner.png
│   ├── foto-produto.png
│   └── logo.png
├── node_modules/
├── package-lock.json
├── package.json
└── src/
    ├── calculo.js
    └── index.js
```

5. Entregáveis

- A sequência completa de comandos usada, do passo 1 ao 8, na ordem.
- Um print da saída de `npm start`.

- Um print da saída de `ls -R loja-lions` (pode cortar a parte do `node_modules`, que é longa).
- A resposta do passo 6.3: para que servem `node_modules` e `package-lock.json` ?

Dica: antes de apagar qualquer coisa com `rm`, rode `ls` com o mesmo padrão (`ls *.tmp`) para conferir quais arquivos serão apagados. O `rm` não manda nada para a lixeira. Nomes com espaço precisam de aspas: `mv "pasta nova" config` .

LionsDev • Professor Nicolas Cardoso Motta

Desafio Prático de Terminal - Módulo 01